

Decentralized Marketplace

A peer-to-peer Ethereum marketplace for digital and physical goods.

Version: 1.0.0 | **Date:** May 2026

Table of Contents

1. Project Overview
2. Features
3. Technology Stack
4. Architecture
5. Smart Contract (MarketPlace.sol)
6. Frontend Application
7. Testing
8. Deployment Guide
9. Usage
10. Screenshots
11. Appendix

1. Project Overview

The Decentralized Marketplace is a full-stack dApp (decentralized application) that allows users to list products for sale and purchase items directly on the Ethereum blockchain. Built with Solidity, Truffle, and React, it removes intermediaries and ensures trustless transactions through smart contracts.

Sellers can create product listings by specifying a name and price (in ETH). Buyers can securely purchase items by sending the exact amount of Ether to the contract, which then transfers ownership and automatically forwards the payment to the seller.

2. Features

- **Create Product** – Anyone can list a new product with a name and price.
- **Purchase Product** – Buy a product by sending Ether; the contract verifies the amount, prevents self-purchase, and handles the transfer.
- **Ownership Transfer** – After purchase, the product's owner is updated to the buyer.
- **Event Logging** – All creation and purchase events are emitted for off-chain tracking.
- **Validation** – Name cannot be empty, price must be > 0 , duplicate purchases and self-buying are prevented.
- **Modern UI** – Clean, responsive React interface with real-time product listing and transaction feedback.

3. Technology Stack

Layer	Technology
Blockchain	Ethereum (Solidity 0.5.0)

Layer	Technology
Development Framework	Truffle 5.0.5
Smart Contract Testing	Mocha, Chai, chai-as-promised
Frontend	React 16.8, Web3.js (beta.55)
UI Styling	Bootstrap 4.3, custom CSS
Wallet	MetaMask
Local Blockchain	Ganache (port 7545)

4. Architecture

The system consists of three main parts:

1. **Smart Contract (MarketPlace.sol)** – Deployed on the Ethereum network, it holds all product data and business logic.
2. **React Frontend** – Provides a user interface for listing and purchasing products. It communicates with the smart contract via Web3.js and MetaMask.
3. **Truffle Suite** – Compiles, deploys, and tests the smart contract.
Data flow: User → MetaMask → Web3.js → Smart Contract (on-chain). Events emitted by the contract are captured in the frontend to refresh the UI.

5. Smart Contract (MarketPlace.sol)

The contract is written in Solidity `^0.5.0` and contains:

5.1 State Variables

Variable	Type	Description
name	string public	Marketplace name ("MarketPlace")
productCount	uint public	Total number of products listed
products	mapping(uint => Product)	Stores products by ID

5.2 Struct

```
struct Product {
    uint id;
    string name;
    uint price;           // in wei
    address payable owner;
    bool purchased;
}
```

5.3 Functions

Function	Visibility	Description
----------	------------	-------------

<code>createProduct(string memory _name, uint _price)</code>	<code>public</code>	Creates a new product. Requires non-empty name and price > 0.
<code>purchaseProduct(uint _id)</code>	<code>public payable</code>	Buys a product. Checks validity, transfers funds, updates ownership.

5.4 Events

- `ProductCreated(uint id, string name, uint price, address owner, bool purchased)`
- `ProductPurchased(uint id, string name, uint price, address owner, bool purchased)`

5.5 Complete Source

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.5.0;

contract MarketPlace {
    string public name;
    uint public productCount = 0;
    mapping(uint => Product) public products;

    struct Product {
        uint id;
        string name;
        uint price;
        address payable owner;
    }
}
```

```

        bool purchased;
    }

    event ProductCreated(
        uint id,
        string name,
        uint price,
        address payable owner,
        bool purchased
    );

    event ProductPurchased(
        uint id,
        string name,
        uint price,
        address payable owner,
        bool purchased
    );

    constructor() public {
        name = "MarketPlace";
    }

    function createProduct(string memory _name, uint _price) public payable {
        require(bytes(_name).length > 0, "Product name cannot be empty");
        require(_price > 0, "Product price must be greater than 0");
        productCount++;
        products[productCount] = Product(
            productCount,
            _name,
            _price,
            msg.sender,
            false
        );
        emit ProductCreated(productCount, _name, _price, msg.sender);
    }

    function purchaseProduct(uint _id) public payable {
        require(_id > 0 && _id <= productCount, "Invalid product ID");
    }

```

```

        Product memory _product = products[_id];
        address payable _seller = _product.owner;
        require(msg.value >= _product.price, "Insufficient Eth");
        require(!_product.purchased, "Already purchased");
        require(_seller != msg.sender, "Cannot buy own product");
        _product.owner = msg.sender;
        _product.purchased = true;
        products[_id] = _product;
        _seller.transfer(msg.value);
        emit ProductPurchased(
            _id,
            _product.name,
            _product.price,
            msg.sender,
            true
        );
    }
}

```

6. Frontend Application

The React app consists of three main components:

- **App.js** – Main component: initialises Web3, loads the smart contract, and manages state (products, account, loading).
- **Navbar.js** – Displays the application title and the connected wallet address.
- **AddProductForm** (inside App.js) – Form to submit new products.
- **ProductList** (inside App.js) – Renders a grid of product cards with buy buttons.

Key features of the frontend:

- Connects to MetaMask and detects the correct network.
- Loads all products from the smart contract on page load.
- Uses a polling mechanism to wait for transaction receipts, ensuring the UI updates instantly after a transaction is mined.

- Provides user feedback via loading spinners and error alerts.
- Styled with a custom, professional CSS theme (no external dependencies).

7. Testing

The project includes a comprehensive test suite written with Truffle's test framework (Mocha + Chai). The tests cover:

- **Deployment** – Contract deploys with the correct name.
- **Product Creation** – Successful creation, invalid name/price rejection.
- **Product Listing** – Retrieving stored product details.
- **Product Purchase** – Successful purchase, event emission, Ether transfer to seller, and various failure cases (invalid ID, insufficient payment, double purchase, self-buy).

To run the tests:

```
truffle test
```

All tests pass successfully against a local Ganache instance.

8. Deployment Guide

1. Install dependencies: `npm install`
2. Start Ganache on **port 7545** (GUI or CLI).
3. Compile and migrate contracts:

```
truffle compile  
truffle migrate
```


4. Copy the generated ABI from `src/abis/MarketPlace.json` (automatically done by Truffle).
5. Start the React development server:

```
npm run start
```

6. Connect MetaMask to `http://localhost:7545` and import a Ganache account.
7. The marketplace is now ready to use.

9. Usage

Listing a product:

1. Enter a product name (e.g., "Vintage Watch") and a price in ETH (e.g., 0.5).
2. Click **Add** and confirm the MetaMask transaction.
3. The product appears immediately in the product grid.

Buying a product:

1. Switch to a different account in MetaMask (buyer).
2. Click the **Buy Now** button on the desired product.
3. Confirm the transaction. The Ether is transferred to the seller, and the product status changes to "Sold".

10. Screenshots

Below are examples of the interface (placeholders).

[Screenshot of the main product list and "Add Product"

form]

[Screenshot of a product after successful purchase]

11. Appendix

The complete ABI (Application Binary Interface) can be found in the `src/abis/MarketPlace.json` file after compilation. The source code for all components (smart contract, tests, frontend) is available in the project repository.
